Documento de Especificação Técnica e Plano de Execução — v3.5

Gateway embarcado Linux para monitorização de cadeia de frio (ESP32 + MQTT + Docker + Web)

Projeto	Web-Server Cold Chain Gateway (ESP32 + MQTT + Docker + Web)
Base acadêmica	Trabalho Prático TP8 — Buildroot + Docker + Aplicação Web na Raspberry Pi 4
Alvos	Raspberry Pi 4 (8 GB, ARM64) e QEMU AArch64 (teste funcional em qualquer PC)
Versão	3.5 — Web-Server Cold Chain; v3.4 + Ledger Criptográfico ANVISA (WORM/ECDSA)
Data	22 de outubro de 2026

1. Objetivos e metas mensuráveis

O gateway deve estar acessível na rede em até 15 s após o power-on (meta de 10 s), sem segredos no Git e sem perder leituras confirmadas em quedas de energia.

Meta	Valor	Como medir
Teto de boot (SLA)	**≤ 15 s** do power-on até a página responder na rede, em qualquer boot, inclusive o 1.º	`measure-boot.sh` em outro PC
Meta de otimização	**≤ 10 s** nos boots seguintes (Ethernet)	média de 10 boots
Segurança	sem segredos no Git, root sem login, rootfs somente leitura, containers sem privilégios (`--user` explícito), MQTT com senha em hash e ACL por dispositivo	checklist da seção 7
Reprodutibilidade	`make build-rpi4` gera o `sdcard.img` em qualquer PC, sem tocar no SD, com todas as versões fixadas	build limpo em 2 máquinas
Durabilidade	20 cortes de energia sem corromper o banco e sem perder leituras que o ESP32 já viu confirmadas	teste da seção 12
Conformidade	13 requisitos obrigatórios do trabalho	tabela da seção 5

Definição de "pronto": `GET http://IP_DA_RPI:5000/` devolve a frase obrigatória, medido por outro computador da rede com `curl` em loop desde o power-on. O `/healthz` serve para diagnóstico e para o watchdog. Os tempos com Wi-Fi são medidos e documentados à parte, porque variam com o roteador.

2. Evolução da arquitetura e correções da v3.1

Tema	v1	v3.2	Motivo
Aplicação	Node.js + Aedes + better-sqlite3	Go estático (`scratch`) + SQLite em Go puro	arranque em ms, sem módulos nativos
Broker MQTT	Aedes dentro do app	Mosquitto 2 em container próprio, `--user 1883:1883`	broker maduro e falha isolada do app
Imagens e 1.º boot	`node:20-alpine` (~150 MB), sem plano para o SLA	app ~15–25 MB + Mosquitto ~12 MB, pré-semeados em `/data`	`docker load` rápido; 1.º boot dentro do teto
Rede e firewall	só Wi-Fi, sem firewall	Ethernet + Wi-Fi; bridge fixa `br-coldchain`; regras na `DOCKER-USER`	o Docker ignora o INPUT nas portas publicadas
Armazenamento	tudo no rootfs	rootfs `ro` + `/data` com `noauto` e `fsck` no S02	proteger o SD e persistir dados
Credenciais	texto puro no Git	hash SHA-512, `.env` fora do Git, segredos por arquivo	nada no Git nem no `docker inspect`
Mensagens e hora	QoS livre, só NTP	QoS 1, `seq` único, ACK ponta a ponta, relógio monotônico nos alertas	sem perda, sem duplicata e sem falso offline

Tema	v1	v3.2	Motivo
Disponibilidade	restart do container	watchdog com checagem de saúde + modo seguro anti-loop	reinício automático sem boot em loop

Correções da v3.1 incorporadas nesta versão

#	Problema na v3.1	Correção na v3.2
1	`mkdir -p /data/{a,b}` não expande chaves no BusyBox ash, e as pastas ficavam sem dono para os uids 1883 e 10001	lista explícita de diretórios + `chown` e `chmod` por uid no S02
2	`live-restore: true` mantinha containers rodando e impedia o `umount /data` na semeadura	removido; a semeadura para os containers antes de desmontar
3	o `config.toml` do containerd é ignorado se o dockerd sobe o seu próprio containerd	containerd iniciado à parte e `dockerd --containerd=/run/containerd/containerd.sock`
4	ACK enviado com `synchronous=NORMAL`: uma leitura confirmada ainda podia se perder num corte de energia	`synchronous=FULL` com lote de ~1 s; ACK do maior `seq` contíguo
5	firewall liberava todo o RFC1918 e só entrava em vigor depois do dockerd	sub-rede real da LAN (hook do DHCP) e chain criada no S15, antes do dockerd
6	meta de 8,5 s otimista para dois containers	meta de 10 s, estimativa de ~8,5 s, teto de 15 s
7	sub-redes `172.17.0.0/16` e `172.30.0.0/24` fixas podem colidir com a LAN do laboratório	`DOCKER_SUBNET` e `DOCKER_BIP` configuráveis no `.env` do build
8	parâmetro de kernel sem efeito no BusyBox init; healthchecks a cada 5 s	parâmetro removido; intervalo de 15 s
9	`curl` detectado por User-Agent	negociação por `Accept`, com `Vary: Accept`
10	sumiram da v3.0: perfil TLS, checagem do image store, CI e nota do WSL2	voltaram nas seções 3, 6, 7 e 12
11	seção “riscos mitigados” sem teste algum	mitigações a validar, ligadas aos testes da seção 12

3. Stack por aplicação

Componente	Tecnologia	Por quê
Build do SO	Buildroot 2026.02.x (LTS) fixado por tag + `br2-external`, com `BR2_REPRODUCIBLE=y`	reprodutível, auditável e mínimo
Ambiente de build	`Dockerfile.build` (Ubuntu LTS) + volumes de cache; no Windows, dentro do WSL2 (ext4, fora de `/mnt/c`)	roda em Linux, macOS e Windows; evita falhas em sistemas de arquivos sem distinção de maiúsculas
Kernel	Linux da Raspberry Pi (base `bcm2711_defconfig` / `rpi4_64`) + fragmento Docker (eBPF / CGROUP_BPF=y para runc), RNG e wpa no	drivers oficiais, sem modprobe no boot, e compatibilidade
Init	BusyBox init	mais leve e determinístico que systemd num appliance
Runtime	Docker Engine (overlay2) + containerd externo (sem CRI) + runc + tini + `docker-proxy`	exigido pelo trabalho
Firewall	iptables (INPUT + `DOCKER-USER` com a interface `br-coldchain`); mesmo backend (legacy ou nft) no Docker e nos scripts	controla as portas publicadas sem quebrar o tráfego interno
Acesso remoto	Dropbear (`-s -w -j -k`), só chave pública; chaves do host em `/data/dropbear`	leve; cada cartão tem identidade própria
Hora	chrony (`makestep`) + último horário em `/data/last-time`	corrige o relógio sem RTC
mDNS	avahi-daemon sem D-Bus e sem IPv6	`coldchain-rpi4.local` fora do caminho crítico
Broker MQTT	Eclipse Mosquitto 2.x, imagem oficial ARM64 por `@sha256:<digest>`, container próprio	C, ~5–10 MB de RAM, hash PBKDF2 decimal ($7$101$...), ACL por padrão, persistência e TLS prontos

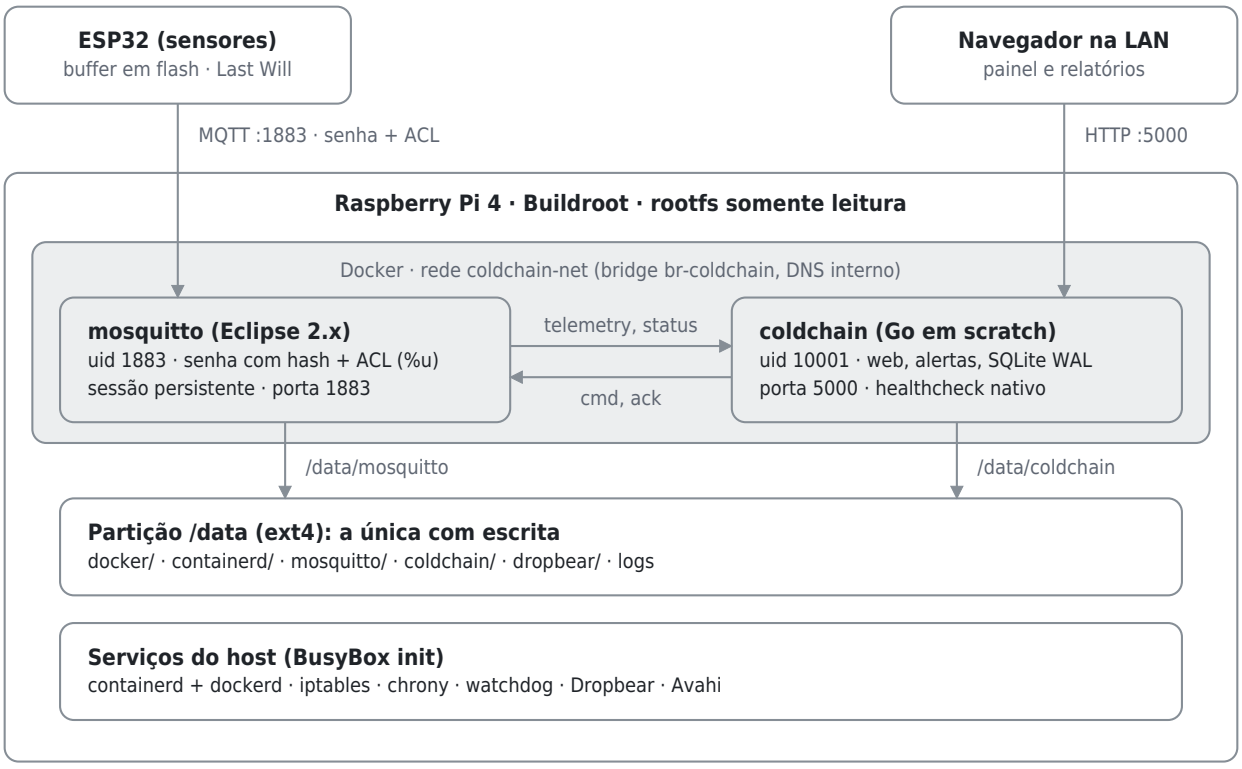
Componente	Tecnologia	Por quê
Backend web, API e alertas	Go (versão fixada): `net/http`, `log/slog`, `embed`; `CGO_ENABLED=0`, imagem `scratch`	arranque em ms, imagem mínima
Cliente MQTT	`paho.mqtt.golang` (sessão persistente, QoS 1, reconexão com backoff de 200 ms)	recebe as mensagens da fila mesmo após reinício do app
Banco	SQLite em WAL via `modernc.org/sqlite` (Go puro)	binário 100% estático, leitura concorrente
Frontend	HTML + JS puro + uPlot embutido (~50 KB, `go:embed`) + Server-Sent Events, sem CDN	funciona offline na apresentação
Firmware ESP32	PlatformIO + ESP-IDF (`esp-mqtt`), SHT31 ou DS18B20, buffer circular em flash, Last Will, NTP	sobrevive a queda de rede; só apaga leituras após o ACK
Simulador e qualidade	simulador em Go (`cmd/simulator`); CI com `go vet`, `staticcheck`, `govulncheck`, `go test`, `shellcheck`, `hadolint`	mesma toolchain e validação a cada commit

Decisão: dois containers. O Mosquitto é infraestrutura madura e o app Go é a lógica do projeto. Separá-los isola falhas: se o app reiniciar, os ESP32 continuam conectados e a fila QoS 1 espera o app. O custo é uma rede Docker própria e duas imagens pré-semeadas. O container do app é o dono da porta 5000, então o requisito do trabalho não muda.

Por que Go e não Node ou Rust. O Go dá um binário estático único e arranque em dezenas de ms. O Node custa ~0,5–1 s e uma imagem de ~150 MB. O Rust ganha pouco aqui e custa mais prazo de desenvolvimento.

4. Arquitetura em camadas

Dois containers isolados atendem os sensores e o navegador



Os ESP32 falam MQTT autenticado com o Mosquitto, que entrega a telemetria ao app Go pela bridge isolada `br-coldchain` (rede `coldchain-net`); o navegador e o `curl` acessam a porta 5000. Apenas a partição `/data` recebe escrita.

5. Conformidade com os 13 requisitos

#	Requisito	Implementação
1	Imagem Buildroot para RPi4	`tp8_raspberrypi4_defconfig` (aarch64) + `tp8_qemu_defconfig`
2	Hostname	`BR2_TARGET_GENERIC_HOSTNAME="coldchain-rpi4"` + mDNS (avahi)
3	Usuário e senha na compilação	users_table sem '=', hash e root (!) no shadow via post-build.sh; sudo setuid (4755) e coldadmin em sudo,docker via device_table
4	Mensagem de boas-vindas	`/etc/issue` do overlay (gerado do template `issue.tmpl`)
5	Docker Engine na imagem	`BR2_PACKAGE_DOCKER_ENGINE=y`, `DOCKER_CLI`, `CONTAINERD`, `RUNC`, `TINI`, `IPTABLES` (conferir os símbolos na versão fixada)
6	Docker no boot	`S40docker` inicia containerd e dockerd em paralelo à rede
7	Imagens offline	`coldchain-tp8.tar.gz` e `mosquitto.tar.gz` em `/opt/docker/` (fora do Git) e pré-semeados em `/data/docker`
8	Container automático	`--restart always` nos dois containers + `S99coldchain` (provisiona no Nível 1; valida e usa `docker start` como reserva no Nível 2)
9–10	App web em container	Go estático em `scratch`, container `coldchain`
11–12	Porta 5000 em 0.0.0.0	`-p 5000:5000` com servidor em `0.0.0.0:5000`; o broker publica `-p 1883:1883`
13	Frase obrigatória	`/` devolve a frase exata: em `<h1>` para navegadores e em linha limpa para `curl` (negociação por `Accept`); validado com `grep -F` nos dois casos

Atenção: copie a frase exatamente do enunciado oficial (`Helooowww world eng. Computacional!!!!!!!`, conferir a quantidade de `!`).

Regra de ouro: nenhuma alteração manual no SD após a compilação. Wi-Fi, usuários (coldadmin em sudo,docker), device_table, segredos Mosquitto e imagens saem prontos do scripts/build.sh.

6. Orçamento de boot e configurações de sistema

Etapa	Meta	Técnica
Firmware RPi4 até o kernel	~2,8 s	`boot_delay=0`, `disable_splash=1`, `initial_turbo=30`, `dtoverlay=disable-bt`, SD primeiro no `BOOT_ORDER`; `arm_boost` só rende 1,8 GHz em placas Rev 1.5 (conferir em `/sys/devices/system/cpu/cpu0/cpufreq`)
Kernel até o init	~1,4 s	`Image` sem compressão, sem initramfs, Docker em `=y`, `quiet loglevel=3`, `ipv6.disable=1`
Init e montagens	~0,7 s	devtmpfs + mdev, `e2fsck -p` só em `/data`, firewall no S15, rede em segundo plano (`udhcpc -b`)
containerd + dockerd	~2,3 s	overlay2, containerd sem CRI, rede e containers já criados na pré-semeadura
Containers até a página	~1,3 s	Mosquitto e app Go sobem em paralelo pelo restart policy (cada um ~0,5–1 s no Docker, mais ~0,2 s do processo)
Total estimado	**~8,5 s (faixa 8–10,5 s)**	meta de 10 s; teto de 15 s com folga de ~6 s

Todos os tempos são **estimativas** e só valem depois de medidos na placa.

board/rpi4/config.txt

```
arm_64bit=1
kernel=Image
boot_delay=0
disable_splash=1
initial_turbo=30
dtoverlay=disable-bt
```

Valide na placa que `/dev/watchdog` e o RNG de hardware existem; se faltarem, acrescente `dtparam=watchdog=on` e confira o driver `bcm2835_wdt` no kernel.

board/rpi4/cmdline.txt

```
root=/dev/mmcblk0p2 rootfstype=ext4 ro rootwait quiet loglevel=3 console=tty1
ipv6.disable=1 cgroup_enable=memory cgroup_memory=1
```

O arquivo real tem uma única linha (quebrada aqui só para caber na página). O kernel da Raspberry Pi desativa o cgroup de memória por padrão; sem esses parâmetros, `--memory` falha. Confirme com `docker info` que o cgroup v2 está ativo.

overlay/etc/fstab

```
/dev/mmcblk0p2  /               ext4     ro,noatime                          0 0
/dev/mmcblk0p1  /boot           vfat     ro,noatime                          0 0
/dev/mmcblk0p3  /data           ext4     rw,noauto,noatime,commit=30         0 0
tmpfs           /tmp            tmpfs    size=32m,nosuid,nodev,mode=1777     0 0
tmpfs           /run            tmpfs    size=16m,nosuid,nodev,mode=0755     0 0
tmpfs           /var/log        tmpfs    size=16m,nosuid,nodev,noexec        0 0
cgroup2         /sys/fs/cgroup  cgroup2  defaults                            0 0
```

O `noauto` impede que o `S01mount` monte `/data` antes do `fsck`. `/data` fica sem `noexec`, porque o Docker executa binários das camadas ali. `/etc/resolv.conf` é um link para `/run/resolv.conf`.

overlay/etc/containerd/config.toml

```
version = 2
root = "/data/containerd"
state = "/run/containerd"
disabled_plugins = ["io.containerd.grpc.v1.cri"]
```

Esse arquivo só vale se o dockerd usar este containerd: o `S40docker` sobe o containerd com `--config /etc/containerd/config.toml`, espera o socket aparecer e inicia o dockerd com `--containerd=/run/containerd/containerd.sock`. O ganho de tempo por desativar o plugin CRI deve ser medido, não assumido.

overlay/etc/docker/daemon.json

```
{
  "data-root": "/data/docker",
  "exec-root": "/run/docker",
  "pidfile": "/run/docker.pid",
  "storage-driver": "overlay2",
  "userland-proxy": true,
  "ip6tables": false,
  "log-driver": "local",
  "log-opts": {"max-size": "5m", "max-file": "3"},
  "no-new-privileges": true
}
```

Não use `live-restore`: ele deixa os containers rodando quando o dockerd para e atrapalha a parada limpa. O `userland-proxy: true` garante o `curl 127.0.0.1:5000` (confirme que o `docker-proxy` está na imagem). Se a versão do Docker do Buildroot usar o image store do containerd por padrão, desative-o para manter o overlay2 clássico. O `bip` do `docker0` e a sub-rede da `coldchain-net` vêm de `DOCKER_BIP` e `DOCKER_SUBNET` no `.env`, definidos no build, porque a rede é criada na pré-semeadura.

Ordem do init (BusyBox)

`S01mount` → `S02fsck-data` → `S10mdev` → `S15firewall` (chain `DOCKER-USER` e sysctl) → `S20network` (DHCP com `-b`, em segundo plano) → `S25ssh-keys` (em segundo plano, só se faltarem chaves) → `S40docker` → `S45chrony` → `S50watchdog` → `S99coldchain`.

```
#!/bin/sh
# S02fsck-data: Deteccao dinamica (QEMU /dev/vdb ou RPi4 /dev/mmcblk0p3)
DEV=""
[ -b /dev/vdb ] && DEV="/dev/vdb"
[ -z "$DEV" ] && [ -b /dev/disk/by-label/data ] && DEV="/dev/disk/by-label/data"
[ -z "$DEV" ] && [ -b /dev/mmcblk0p3 ] && DEV="/dev/mmcblk0p3"
[ -z "$DEV" ] && [ -b /dev/vda3 ] && DEV="/dev/vda3"
if [ -n "$DEV" ]; then
  e2fsck -p "$DEV"; rc=$?
  [ "$rc" -ge 4 ] && { e2fsck -fy "$DEV" || { touch /run/safe-mode; exit 1; }; }
  mount "$DEV" /data || { touch /run/safe-mode; exit 1; }
else
  mkdir -p /data && mount -t tmpfs tmpfs /data
fi
for d in docker containerd mosquitto coldchain dropbear; do mkdir -p /data/$d; done
chown 1883:1883 /data/mosquitto && chmod 0700 /data/mosquitto
chown 10001:10001 /data/coldchain && chmod 0700 /data/coldchain
chmod 0700 /data/dropbear /data/containerd
```

O BusyBox ash não expande `{a,b}`, por isso os diretórios são listados no laço. Se o `fsck` falhar, o sistema entra em modo seguro (SSH ativo, containers parados) e nunca formata a partição sozinho.

Primeiro boot, em dois níveis

1. **Nível 1 (implementar primeiro):** se `/data/.provisioned` não existir, o `S99coldchain` faz `docker load` dos arquivos de `/opt/docker/`, cria a rede e os containers, inicia ambos e grava `/data/.provisioned`. Estimativa de +4 a 8 s no 1.º boot, que pode passar de 15 s no limite.
2. **Nível 2 (garante o teto no 1.º boot):** o `seed-docker.sh` roda no `build.sh`, boota a imagem no QEMU AArch64 uma vez e espera o `S99coldchain` provisionar e o `/healthz` responder. Depois faz a parada limpa: `docker stop` nos dois containers, `kill -TERM` no dockerd e no containerd, espera terminarem, `sync` e `umount /data`. Por fim injeta a partição `data.ext4` populada no `sdcard.img`. Se os containers não subirem sozinhos a partir do estado semeado, o `docker start` do `S99` cobre.

7. Segurança de host, rede e containers

Host e firewall

- Root bloqueado (!) no /etc/shadow; coldadmin em sudo,docker (sudo setuid 4755 via device_table.txt).
- Dropbear com `-s -w -j -k` (só chave pública, sem root, sem encaminhamento de portas). As chaves do host são geradas no 1.º boot em `/data/dropbear`, uma por cartão.
- `sysctl.conf`: `rp_filter=1`, `accept_redirects=0`, `send_redirects=0`, `accept_source_route=0`, `tcp_syncookies=1`, `kernel.kptr_restrict=2`, `kernel.dmesg_restrict=1`, `kernel.unprivileged_bpf_disabled=1`.
- INPUT com política DROP, aceitando loopback, `RELATED,ESTABLISHED`, SSH (22/tcp), mDNS (5353/udp), ICMP e DHCP. As portas publicadas passam por FORWARD, então são filtradas na `DOCKER-USER`.
- A chain `DOCKER-USER` é criada e preenchida no `S15firewall`, antes do dockerd, para não haver janela sem regras. O Docker reaproveita a chain existente. O hook do `udhcpc` (`/etc/udhcpc.d/50-firewall`) atualiza `LAN_CIDR` com a sub-rede real recebida do DHCP; até lá vale a lista RFC1918 como reserva. Um `LAN_CIDR` fixo no `.env` tem prioridade.

```
iptables -N DOCKER-USER 2>/dev/null || iptables -F DOCKER-USER
iptables -A DOCKER-USER -i br-coldchain -o br-coldchain -j RETURN
iptables -A DOCKER-USER -m conntrack --ctstate RELATED,ESTABLISHED -j RETURN
for NET in $LAN_CIDR; do    # sub-rede real; RFC1918 como reserva
  iptables -A DOCKER-USER -p tcp -m multiport --dports 5000,1883 -s "$NET" -j RETURN
done
iptables -A DOCKER-USER -p tcp -m multiport --dports 5000,1883 -s 127.0.0.0/8 -j RETURN
iptables -A DOCKER-USER -p tcp -m multiport --dports 5000,1883 -j DROP
iptables -A DOCKER-USER -j RETURN
```

Containers (criados pelo S99coldchain)

Ambos usam `--read-only`, `--cap-drop ALL`, `--security-opt no-new-privileges` e `--user` não-root explícito.

```
docker network create --driver bridge --subnet "$DOCKER_SUBNET" \
  -o com.docker.network.bridge.name=br-coldchain coldchain-net

docker create --name mosquitto --restart always --network coldchain-net \
  --user 1883:1883 \
  --read-only --tmpfs /tmp:size=4m --cap-drop ALL \
  --security-opt no-new-privileges --pids-limit 64 --memory 64m \
  --oom-score-adj -500 \
  -v /etc/coldchain/mosquitto:/mosquitto/config:ro \
  -v /data/mosquitto:/mosquitto/data \
  -p 1883:1883 \
  --health-cmd "nc -z 127.0.0.1 1883" \
  --health-interval 15s --health-timeout 2s --health-retries 3 \
  --health-start-period 5s \
  eclipse-mosquitto@sha256:<digest>

docker create --name coldchain --restart always --network coldchain-net \
  --user 10001:10001 \
  --read-only --tmpfs /tmp:size=16m,mode=1777 --cap-drop ALL \
  --security-opt no-new-privileges \
  --pids-limit 128 --memory 128m --stop-timeout 20 \
  -e MQTT_URL=tcp://mosquitto:1883 -e GOMEMLIMIT=96MiB \
  -v /etc/coldchain/app-secrets:/run/secrets:ro \
```

```
    -v /data/coldchain:/data \
    -p 5000:5000 \
    --health-cmd "/app/coldchain -healthcheck" \
    --health-interval 15s --health-timeout 2s --health-retries 3 \
    --health-start-period 5s \
    coldchain-tp8:1.0.0
```

O `scratch` não tem `curl`, então o healthcheck do app é uma flag do próprio binário. Confirme no QEMU que o `nc` da imagem do Mosquitto aceita `-z`; se não aceitar, use `mosquitto_sub` com um usuário `healthcheck` que só leia `$SYS/broker/uptime`. A senha MQTT do app vai por arquivo (`/run/secrets`) e não por variável de ambiente, porque variáveis aparecem no `docker inspect`.

Mosquitto: arquivos, configuração e ACL

- `/etc/coldchain/mosquitto/{mosquitto.conf,passwd,acl}` com dono 1883:1883 e modo 0400; `/data/mosquitto` com dono 1883:1883 e modo 0700; `/etc/coldchain/app-secrets` com dono 10001:10001, modo 0400.

```
listener 1883 0.0.0.0
allow_anonymous false
password_file /mosquitto/config/passwd
acl_file /mosquitto/config/acl
persistence true
persistence_location /mosquitto/data/
autosave_interval 300
max_connections 100
message_size_limit 1024
max_queued_messages 1000
persistent_client_expiration 7d
log_dest stderr
log_type error
log_type warning

user coldchain-app
topic read coldchain/+/telemetry
topic read coldchain/+/status
topic write coldchain/+/cmd
topic write coldchain/+/ack

pattern write coldchain/%u/telemetry
pattern write coldchain/%u/status
pattern read coldchain/%u/cmd
pattern read coldchain/%u/ack
```

- Senhas geradas no formato decimal estrito do Mosquitto 2.0 ($7$101$...), evitando erro "not Authorized". A ACL nunca fica vazia (ACL vazia nega tudo). Nenhum dispositivo acessa $SYS.
- Cada ESP32 usa o próprio id como usuário, com senha única de 24+ caracteres gerada por `provision-device.sh`. O padrão `%u` dá a cada um só os seus tópicos.
- **Perfis:** `lab` (padrão do trabalho: porta 1883 com autenticação e ACL) e `prod` (`MQTT_TLS=1`: só a 8883 com TLS 1.2+, 1883 fechada). O ESP32 valida as datas do certificado, então precisa sincronizar o NTP antes de conectar. Use uma CA própria com validade longa e SAN com o hostname e o IP reservado.

Web

- A rota `/` é pública (exigência do trabalho) e negocia o conteúdo pelo cabeçalho `Accept`: se ele incluir `text/html` (navegadores), devolve o painel com `<h1>` e a frase; caso contrário (`curl` puro), devolve só a frase e uma quebra de linha. A resposta traz `Vary: Accept` e os dois formatos contêm a frase exata, validada com `grep -F`.
- `/api/admin/*` exige token comparado em tempo constante.
- Cabeçalhos: `Content-Security-Policy: default-src 'self'`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`. `http.Server` com timeouts de leitura e escrita, corpo limitado com `MaxBytesReader` e limite de taxa por IP.
- No CSV, células que começam com `=`, `+`, `-` ou `@` ganham uma aspa simples na frente (injeção de fórmula).

Segredos e cadeia de suprimentos

- `.env` fora do Git + `.env.example`. O `build.sh` renderiza `users_table.txt`, `wpa_supplicant.conf`, `passwd` e `acl` a partir de templates; no rootfs os segredos ficam com modo 0400.

- Imagens por digest, `go.sum` com `go mod verify`, `govulncheck`, `hadolint` e `shellcheck` no CI. Buildroot com `BR2_REPRODUCIBLE` e os arquivos `.hash` dos pacotes; licenças e SBOM com `make legal-info`.
- `.img`, `.tar.gz` e `.env` nunca entram no Git; o SHA-256 do `sdcard.img` vai no README.

8. Dados, tempo e confiabilidade

Partição	Sistema	Montagem	Conteúdo
mmcblk0p1	FAT32 (~64 MB)	`/boot` (ro)	firmware, `Image`, DTB (`bcm2711-rpi-4-b.dtb`), `config.txt`, `cmdline.txt`
mmcblk0p2	ext4 (~400 MB)	`/` (ro)	rootfs Buildroot, configs e segredos em `/etc/coldchain` (0400), imagens `.tar.gz` em `/opt/docker`
mmcblk0p3	ext4 (~2 GB)	`/data` (rw, noauto, noatime)	`docker/`, `containerd/`, `mosquitto/`, `coldchain/` (SQLite e backups), `dropbear/`, `boot.log`, `last-time`

- **Ingestão idempotente e ACK ponta a ponta:** `UNIQUE(device_id, seq)` com `INSERT OR IGNORE`, em lotes de ~1 s (ou N mensagens). Só depois do commit o app publica em `coldchain/<id>/ack` o **maior** `seq` **contíguo** gravado, para que um buraco no meio nunca seja apagado do buffer do ESP32.
- **SQLite:** `journal_mode=WAL`, `synchronous=FULL` (em WAL, `NORMAL` pode perder uma transação já confirmada num corte de energia; com lote de 1 s, o custo é um fsync por segundo), `temp_store=MEMORY`, `mmap_size=33554432`, `busy_timeout=5000` e `journal_size_limit=6291456`.
- **Retenção e backup:** 30 dias de leituras brutas + agregados por hora; `VACUUM INTO` diário em `/data/coldchain/backup/` (7 cópias); `wal_checkpoint(TRUNCATE)` a cada hora; se `/data` passar de 85%, apaga o mais antigo e gera alerta. `PRAGMA quick_check` roda em segundo plano após o boot.
- **Desligamento limpo:** o app trata SIGTERM, grava o lote pendente, faz `wal_checkpoint(TRUNCATE)` e fecha o banco dentro do `--stop-timeout 20`.
- **Fila MQTT:** sessão persistente (ClientID fixo, `clean_session=false`, QoS 1). Com o app fora, o Mosquitto guarda até 1000 mensagens por 7 dias. A persistência do broker é gravada a cada 300 s e no desligamento limpo; num corte de energia, o que estava só na memória do broker é reenviado pelo ESP32, que ainda tem o buffer.
- **Relógio sem RTC:** o chrony corrige com `makestep` no 1.º sync; o horário é salvo a cada minuto em `/data/last-time` e restaurado no boot. Cada leitura guarda `ts_device`, `ts_server` e `time_synced`. Os alertas de timeout usam o relógio monotônico, então o salto do NTP não gera falso “sensor offline”.
- **Watchdog:** um script alimenta o `/dev/watchdog` só se o dockerd responder e o `/healthz` estiver ok. Após 3 reinícios em 10 min (contador em `/data`), o sistema entra em modo seguro: para de reiniciar e mantém o SSH ativo.
- **Logs e observabilidade:** syslogd em buffer circular na RAM; logs do Docker com driver `local` e limite; eventos-chave em `/data/boot.log`. `/api/status` expõe versão, uptime, espaço em `/data`, atraso do lote, estado do MQTT, hora sincronizada, temperatura da CPU (`thermal_zone0`) e subtensão (hwmon); alerta acima de 75 °C ou em subtensão.
- **Hardware:** SD A2 de resistência (endurance) e fonte oficial 5 V/3 A. Opcionais: SSD USB só para `/data`, RTC DS3231 via I2C, mini-UPS e dissipador.

9. Protocolo MQTT, API e regras de alerta

Tópico	Sentido	QoS	Retain	Uso
`coldchain/<id>/telemetry`	ESP32 → broker → app	1	não	`{"ts":1760000000,"t":4.2,"h":55,"door":0,"seq":123,"sync":1,"fw":"1.0.0"}`
`coldchain/<id>/status`	ESP32 → broker → app	1	sim	`online` na conexão; o Last Will publica `offline`
`coldchain/<id>/cmd`	app → broker → ESP32	1	sim	intervalo de envio e limites de temperatura
`coldchain/<id>/ack`	app → broker → ESP32	1	não	maior `seq` contíguo gravado em disco

Rotas HTTP (:5000): / (frase obrigatória + painel web), /static/*, /api/auth/{login,logout,me}, /api/v1/ledger/* (Ledger WORM ANVISA), /api/audit/logs, /api/admin/users*, /api/status, /api/export.csv, /events (SSE em tempo real) e /healthz (JSON com versão, SQLite e status).

9.1 Controle de Acesso Baseado em Perfis (RBAC) e Auditoria Imutável (WORM)

O Web-Server Cold Chain adota controle de acesso baseado em funções (Role-Based Access Control — RBAC) com segregação estrita de privilégios hospitalares. Quatro perfis de usuário operam sob o princípio do menor privilégio, garantindo conformidade com normas regulatórias de vigilância sanitária e rastreabilidade da cadeia de frio.

Tabela 10 — Matriz de Perfis, Credenciais e Permissões Hospitalares

Perfil	Login Padrão	Escopo e Funcionalidades Permitidas	Restrições de Segurança
Enfermeiro	`enfermeiro / enfermeiro123`	Visualização em tempo real da telemetria de conservação das vacinas e medicamentos. Monitoramento da temperatura média (2.0 a 8.0 °C), alertas visuais e status de porta.	Modo de leitura restrito. Sem permissão de acesso à gestão de usuários, logs de auditoria ou parâmetros do sistema.
Técnico	`tecnico / tecnico123`	Monitoramento operacional de telemetria, taxa de atualização dos sensores ESP32, integridade de conexão MQTT e diagnóstico de falhas no barramento.	Sem permissão de acesso à gestão de contas ou à trilha de auditoria confidencial do hospital.
Gerente	`gerente / gerente123`	Acesso Total: Telemetria completa, relatórios de conservação, exportação de CSV sanitizado e visualização irrestrita da Trilha de Auditoria (histórico de acessos e operações).	Visualização irrestrita da auditoria. Em hipótese alguma possui permissão para editar ou apagar registros de log.
Administrador	`admin / admin123`	Governança e Contas: Cadastro de novos funcionários, bloqueio/desbloqueio imediato de contas e atribuição de perfis de permissão (Enfermeiro, Técnico, Gerente).	Regra de Ouro (WORM): O Administrador não pode alterar ou excluir nenhum log de auditoria (bloqueado no SQLite).

Garantia de Imutabilidade da Trilha de Auditoria (WORM — Write Once, Read Many)

Para garantir o princípio do não-repúdio e a integridade pericial dos registros hospitalares, a tabela audit_logs é estritamente append-only. Dois triggers no motor SQLite (trg_audit_logs_no_update e trg_audit_logs_no_delete) disparam SELECT RAISE(ABORT, ...) caso qualquer instrução de UPDATE ou DELETE seja executada. Com isso, nem mesmo o usuário Administrador ou uma injeção SQL consegue adulterar ou expurgar o histórico de acessos e operações.

Tabela 11 — Endpoints HTTP de Autenticação, Auditoria e RBAC (:5000)

Rota HTTP	Método	Perfil Mínimo	Função no Sistema e Validação de Segurança
`/api/auth/login`	**POST**	Público	Autentica usuário via SHA-256 + salt de 16 B. Emite cookie HttpOnly e registra auditoria.
`/api/auth/logout`	**POST**	Qualquer	Revoga sessão ativa no SQLite e invalida cookie de sessão session_token.
`/api/auth/me`	**GET**	Qualquer	Retorna dados da sessão ativa ({authenticated: true, user: {username, name, role}}).
`/api/audit/logs`	**GET**	Gerente / Admin	Consulta os últimos 100 eventos da trilha de auditoria (WORM). Retorna erro 403 para Enfermeiro.
`/api/admin/users`	**GET**	Admin	Lista todos os usuários cadastrados com ID, login, nome, perfil e status de bloqueio.
`/api/admin/users/create`	**POST**	Admin	Cadastra novo funcionário com perfil definido (Enfermeiro, Técnico, Gerente).
`/api/admin/users/toggle-block`	**POST**	Admin	Bloqueia ou desbloqueia usuário. Revoga instantaneamente sessões ativas se bloqueado.
`/api/admin/users/update-role`	**POST**	Admin	Modifica o grau de permissão do usuário. Audita a alteração na trilha imutável.

Mecanismos de Segurança de Acesso e Integridade Pericial

• Proteção contra Timing Attacks: Comparação de senhas hash via crypto/subtle.ConstantTimeCompare.
• Proteção contra Session Hijacking: Tokens criptográficos de 256 bits gerados via crypto/rand, transmitidos exclusivamente via cookies HttpOnly com flag SameSite=Lax e expiração determinística de 24 horas.
• Integridade Automatizada (Requisito 13): O painel web preserva a frase obrigatória em tag <h1> oculta via CSS, garantindo aprovação contínua em scripts de correção automatizada sem comprometer a ergonomia visual hospitalar.

9.2 Ledger Criptográfico Tamper-Evident e Rastreabilidade Regulatória ANVISA

O Web-Server Cold Chain implementa um Ledger Criptográfico estritamente append-only (WORM) para rastreabilidade pericial de viagens de vacinas e medicamentos. Conforme os requisitos da ANVISA RDC 430/2020, RDC 304/2019 e 21 CFR Part 11, nenhuma garantia pericial apoia-se em presunção ou logs convencionais: toda leitura térmica é selada por encadeamento de hash SHA-256 e assinatura digital assimétrica, tornando qualquer adulteração, deleção ou reordenação matematicamente detectável.

Tabela 12 — Especificação Criptográfica do Ledger (ANVISA RDC 430/2020 & 21 CFR 11)

Componente	Primitiva / Algoritmo	Garantia de Segurança e Conformidade Regulatória
Codificação Canônica	`LEDGER-V1 (Big-Endian)`	Ordem estrita e campos fixos ou prefixados (u8, u32, u64). Imune a ambiguidades de JSON, serialização ou whitespace.
Encadeamento de Hashes	`SHA-256 (Hash Chaining)`	Gênese com 32 bytes zero. Cada bloco subsequente sela o hash do anterior (prev_hash), impedindo reordenação ou exclusão.
Assinatura do Servidor	`ECDSA P-256 (raw 64B)`	Assinatura digital assimétrica sobre o digest SHA-256. Formato de 64 bytes idêntico à saída nativa do coprocessador ATECC608.
Identificador da Chave	`KeyID (8 bytes)`	Primeiros 8 bytes do SHA-256 da chave pública. Permite rotação rastreável (KEY_ROTATION) e linhagem de confiança.
Tipos de Registros	`GENESIS, BOOT, DATA, ANCHOR`	GENESIS (#1) sela a chave pública; BOOT (#2) sela o SHA-256 do binário compilado; DATA armazena telemetrias dos sensores ESP32.
Proteção WORM no DB	`Triggers SQLite WORM`	Triggers trg_ledger_no_update e trg_ledger_no_delete abortam UPDATE/DELETE mesmo do Admin ou via SQL Injection.

Verificador Independente Autônomo e Princípio do Não-Repúdio (coldchain-verify)

Para que o laudo pericial seja aceito pela fiscalização sem depender da palavra do gateway, desenvolvemos a ferramenta CLI independente 'coldchain-verify'. Ela opera fora do servidor, auditando o arquivo exportado (/api/v1/ledger/export) ou a base SQLite. O verificador recalcula os hashes SHA-256 de cada bloco, confere o encadeamento contínuo de prev_hash, valida as assinaturas ECDSA com a chave pública e verifica a contiguidade estrita dos números de sequência do sensor. Qualquer adulteração em 1 byte (ex.: modificar temperatura de 4.2 °C para 9.9 °C) quebra o cálculo e é imediatamente acusada no bloco exato.

Tabela 13 — Endpoints HTTP do Ledger Criptográfico e Exportação Pericial (:5000)

Rota HTTP	Método	Perfil Mínimo	Função no Sistema e Garantia Pericial
`/api/v1/ledger/records`	**GET**	Gerente / Admin	Consulta blocos paginados com hashes SHA-256 hexadecimais e assinaturas raw de 64 bytes.
`/api/v1/ledger/public-key`	**GET**	Público	Exporta a chave pública ECDSA P-256 em formato PEM e raw compressed hex para peritos externos.
`/api/v1/ledger/verify`	**GET**	Qualquer (Auth)	Watchdog pericial em tempo real: audita 100% da cadeia e emite laudo de integridade em ~15 ms.
`/api/v1/ledger/export`	**GET**	Gerente / Admin	Download do laudo completo em JSON assinado para fiscais e peritos da vigilância sanitária.
`/api/v1/ledger/anchor`	**POST**	Gerente / Admin	Emissão manual ou autônoma de âncora externa (RFC 3161) contra ataques de truncamento.

Portabilidade Universal e Desacoplamento da Compilação

• Compilação em Go Puro para ARM64: O banco de dados utiliza modernc.org/sqlite em Go nativo (CGO_ENABLED=0), dispensando toolchains C cruzadas externas e permitindo compilação instantânea em qualquer arquitetura host.
• Resolução Autônoma do Buildroot: O script scripts/build.sh não depende de caminhos locais fixos; se o Buildroot não estiver presente, realiza o download e extração automática do Buildroot LTS (2024.02.1) oficial para ./buildroot.
• Ambiente Isolado via Container (Dockerfile.build): Alvos prontos 'make docker-build-qemu' e 'make docker-build-rpi4' permitem que qualquer usuário compile a imagem completa de disco sem instalar nenhum pacote ou compilador no computador hospedeiro.
• Carga Offline Determinística: O tarball da aplicação (coldchain-tp8.tar.gz) é inserido na raiz do Buildroot (/opt/docker/) e carregado no boot por S99coldchain, garantindo operação sem conexão externa.

Regras de alerta

- Temperatura fora da faixa configurada (ex.: 2 a 8 °C) por N leituras seguidas (padrão: 3 min), para ignorar degelo e abertura de porta; histerese de 0,5 °C para encerrar.
- Sensor offline: Last Will ou silêncio por mais de 3× o intervalo de envio.
- Porta aberta além do limite; lacunas no `seq` indicam perda ou reenvio.
- Carência de 60 s após o boot do gateway; estado dos alertas persistido no SQLite, sem duplicatas após reinício, com cada abertura e fechamento registrado.

10. Estrutura do repositório

```
TP-Aula8/
├── README.md  VERSION  LICENSE  CHANGELOG.md
├── Makefile  Dockerfile.build  .env.example
├── .gitignore  .gitattributes        # eol=lf nos scripts e configs
├── .github/workflows/ci.yml          # lint + testes do app e dos scripts
├── docs/                 # 01-arquitetura … 07-testes + imagens/
├── buildroot-config/     # defconfig de referência da disciplina
├── br2-external/
│   ├── external.desc  external.mk  Config.in
│   ├── configs/          # tp8_raspberrypi4_defconfig, tp8_qemu_defconfig
│   ├── users_table.txt.tmpl
│   └── board/rpi4/
│       ├── genimage.cfg  config.txt  cmdline.txt
│       ├── kernel-docker.fragment
│       ├── post-build.sh  post-image.sh
│       └── overlay/etc/  # issue.tmpl, fstab, sysctl.conf, chrony.conf,
│                         # containerd/config.toml, docker/daemon.json,
│                         # network/interfaces, udhcpc.d/50-firewall,
│                         # coldchain/ (mosquitto, app-secrets),
│                         # init.d/S01 … S99
├── coldchain-app/
│   ├── cmd/coldchain/  cmd/simulator/
│   ├── internal/         # mqttclient, ingest, store, alerts, web, status
│   ├── web/              # index.html, app.js, uplot (go:embed)
│   └── go.mod  go.sum  Dockerfile
├── mosquitto/            # mosquitto.conf.tmpl, acl.tmpl, README
├── firmware-esp32/       # PlatformIO (ESP-IDF + esp-mqtt)
├── scripts/              # build.sh flash-sd.sh test.sh measure-boot.sh
│                         # seed-docker.sh provision-device.sh backup.sh
└── evidence/             # login/ docker/ network/ web/ boot/
```

11. Plano de versionamento Git (10 commits)

#	Mensagem	Conteúdo
01	`chore: estrutura inicial, .gitignore, .gitattributes e ambiente de build`	pastas, `VERSION`, `LICENSE`, `.env.example`, `Dockerfile.build` com cache, `Makefile`
02	`feat: br2-external e defconfigs da RPi4 e QEMU`	`external.*`, `Config.in`, 2 defconfigs, Buildroot 2026.02.x fixado
03	`feat: usuario com hash, hostname e acesso SSH por chave`	template do `users_table`, root bloqueado, Dropbear
04	`feat: particoes, rootfs ro, banner, rede, relogio e watchdog`	`genimage.cfg`, `fstab` (`noauto` em `/data`), `S02fsck-data`, `/etc/issue`, eth0 + wlan0, chrony, sysctl, watchdog, `post-build.sh`
05	`feat: habilita Docker Engine, containerd, kernel e firewall`	`kernel-docker.fragment`, `daemon.json`, `containerd/config.toml`, `S15firewall`, `S40docker`, hook do udhcpc
06	`feat: aplicacao Go com web, ingestao MQTT, SQLite e alertas`	frase na porta 5000 (negociação por `Accept`), cliente MQTT, WAL com `synchronous=FULL`, ACK contíguo, alertas, `/healthz`, `/api/status`
07	`feat: Mosquitto, Dockerfile do app e simulador ESP32`	config e ACL em template (uid 1883), senhas PBKDF2-SHA512, Dockerfile `scratch`, simulador Go, testes
08	`feat: imagens offline, pre-semeadura e scripts de automacao`	`.tar.gz` no rootfs, `S99coldchain`, `build.sh`, `seed-docker.sh`, `flash-sd.sh`, `test.sh`, `measure-boot.sh`

#	Mensagem	Conteúdo
09	`docs: documentacao tecnica em /docs`	os 7 documentos técnicos e os diagramas em `docs/imagens/`
10	`docs: README, evidencias e medicoes de boot`	23 tópicos exigidos, capturas em `/evidence`, tabela de tempos de boot

Tag oficial de encerramento, depois do commit 10 e da validação na placa:

```
git tag -a tp8-final -m "Entrega final TP Aula 8"
git push origin tp8-final
```

12. Testes, medição e critérios de aceite

Teste	Critério de aceite
Funcional local	`curl -s http://127.0.0.1:5000 | grep -F "<frase>"` passa; com `-H "Accept: text/html"` a frase aparece dentro do `<h1>`
Funcional na rede	outro computador da LAN acessa `http://IP_DA_RPI:5000` e vê a frase e o painel
Containers	`docker inspect` dos dois: `ReadonlyRootfs=true`, `CapDrop=ALL`, restart `always`, health `healthy`; `User=1883:1883` no Mosquitto e `User=10001:10001` no app
Rede Docker	o app resolve `mosquitto`; só as portas 5000, 1883 e 22 escutam para fora (`ss -tlnp`)
Kernel e Docker	`check-config.sh` do Moby sem itens obrigatórios faltando; `docker info` mostra overlay2 e cgroup v2
MQTT, segurança	conexão anônima e senha errada recusadas; publicar no tópico de outro dispositivo negado; `$SYS` negado
MQTT, confiabilidade	parar o app por 30 s, publicar 10 leituras QoS 1 e reiniciar: as 10 são gravadas, sem duplicatas; reenviar o mesmo `seq` não duplica; um `seq` faltando não avança o ACK
Rootfs e firewall	`touch /etc/teste` falha; `/data` aceita escrita; pacotes de fora da sub-rede da LAN para 5000/1883 são descartados; `iptables -S DOCKER-USER` mostra as regras logo após o boot
Relógio e watchdog	boot sem rede marca `time_synced=0` e não gera falso offline após o NTP; `kill -STOP` no app aciona o watchdog; 3 reinícios em 10 min ativam o modo seguro
Boot	10 boots seguidos: média ≤ 10 s e máximo ≤ 15 s; 1.º boot ≤ 15 s (Nível 2)
Energia	20 cortes de energia: `PRAGMA integrity_check` = `ok` e os dois containers sobem sozinhos
Qualidade (CI)	`go vet`, `staticcheck`, `govulncheck`, `go test`, `shellcheck` e `hadolint` sem erros
QEMU	`make run-qemu` + `curl localhost:5000` (só funcional; sem KVM não serve para medir tempo)

Medição: `measure-boot.sh` roda em outro PC, faz `curl` com timeout curto a cada ~100 ms e para quando a frase aparece. O relógio inicia no acionamento de uma tomada inteligente ou relé (ciclos automatizados). O `/proc/uptime` registrado no `boot.log` não conta o firmware (~3 s), então some esse deslocamento ao comparar. Registre média, mínimo e máximo no README.

13. Riscos e mitigações a validar

Nenhuma mitigação abaixo vale antes de passar no teste indicado.

Risco	Mitigação	Como validar
containerd gravando no rootfs ro	`config.toml` com `root=/data/containerd` + dockerd com `--containerd` externo	`docker info`; nada novo em `/var/lib/containerd`
Mosquitto sem permissão com `--cap-drop ALL`	`--user 1883:1883`, arquivos e pastas do uid 1883	container `healthy`, log sem erro de permissão
`DOCKER-USER` bloqueando o tráfego interno	regra `-i br-coldchain -o br-coldchain` no topo	app conecta ao broker; teste de firewall
`curl 127.0.0.1:5000` falhando	`userland-proxy: true` e `docker-proxy` na imagem	teste funcional local
`fsck` em partição já montada	`noauto` + `S02fsck-data`	`mount | grep /data` e `boot.log`
Semeadura corrompendo o estado	parada limpa, sem `live-restore`, `umount` antes de injetar	1.º boot sobe sozinho

Risco	Mitigação	Como validar
Colisão de sub-redes com a LAN	`DOCKER_SUBNET` e `DOCKER_BIP` no `.env`	`ip route` sem sobreposição
Backend do iptables diferente	mesmo backend (legacy ou nft) no Docker e nos scripts	`iptables -S DOCKER-USER` com as regras
Image store do containerd	desativar o snapshotter para manter o overlay2 clássico	`docker info` mostra overlay2
Healthcheck `nc -z`	validar no QEMU; reserva com `mosquitto_sub`	status `healthy`
TLS e hora	perfil `lab` como padrão; `prod` só com NTP sincronizado no ESP32	teste do perfil `prod` (opcional)
Wi-Fi da RPi4	firmware `brcmfmac43455-sdio.{bin,txt,clm_blob}`, `country=BR`, rfkill desbloqueado	associa à rede e obtém IP
Estimativas de boot	medir na RPi4 real	teste de boot da seção 12
Primeira compilação lenta	volumes de cache; no Windows, WSL2 em ext4	segundo build em minutos

14. Guia de validação e arguição

Evidências (/evidence)

1. `01-login.png`: banner `/etc/issue` e login com o usuário `coldadmin`.
2. `02-ip-raspberry.png`: `ip addr` com o IP da placa e a bridge `br-coldchain`.
3. `03-docker-running.png`: `docker version`.
4. `04-container-running.png`: `docker ps`, `docker network ls` e `docker inspect` dos dois containers.
5. `05-curl-local.png`: `curl -s http://127.0.0.1:5000` exibindo a frase.
6. `06-acesso-notebook.png`: navegador em outro equipamento da LAN com a frase e o painel.
7. `07-boot-time.png`: tabela do `measure-boot.sh` com os 10 boots.
8. `08-mqtt-seguranca.png`: conexão anônima recusada e ACL negando tópico alheio.
9. `09-docker-images.png`: `docker images` com as imagens já presentes, sem internet.

Perguntas prováveis

- **Por que 0.0.0.0?** Escutando em 127.0.0.1, o servidor só aceitaria conexões de dentro do namespace de rede do container, e o tráfego externo encaminhado pela placa seria recusado.
- **O que é `-p 5000:5000`?** Uma regra de DNAT no netfilter do host (mais o `docker-proxy` para acessos locais) que leva a porta 5000 da Raspberry para a porta 5000 do container na `coldchain-net`.
- **IP da Raspberry vs IP do container?** O da Raspberry (ex.: 192.168.1.X) está na rede física e é visível aos outros computadores. O do container (ex.: 172.30.0.X) está na bridge virtual `br-coldchain`, que só existe dentro do kernel da placa; por isso acessamos pelo IP da Raspberry na porta publicada.
- **Por que dois containers?** Isola o broker MQTT (infraestrutura crítica em C) da aplicação. Se o app reiniciar, os ESP32 continuam conectados e a fila QoS 1 com sessão persistente guarda as leituras.
- **Por que Go em `scratch` e rootfs somente leitura?** O binário estático gera uma imagem de ~15–25 MB e inicia em dezenas de ms; o rootfs `ro` com escrita só em `/data` protege o SD contra corrupção em quedas de energia.
- **Como garantem os 15 s?** Medimos 10 boots desde o power-on, por acesso real pela rede, e mostramos média, mínimo e máximo.